

Interface de suivi d'accès serveur



Documentation Globale

Cristiano Henrique Gaspar

1. Code C# et registre

1.1 Rôle de l'exécutable testbureau.exe

L'exécutable BureauDistance.exe fait office de pont sécurisé entre l'interface PHP et le client RDP Windows (mstsc.exe).

Pourquoi cette approche ?

- PHP ne peut pas lancer directement mstsc.exe à cause des restrictions UAC Windows
- L'exécutable C# agit comme intermédiaire
- Communication via argument ligne de commande simple

Flux de traitement :

1. Récupère l'adresse serveur depuis l'interface PHP
(*rdpportal://192.192.192.192*)
2. Nettoie l'entrée (garde uniquement l'IP)
3. Lance mstsc.exe /v:IP

1.2 Fichier registre rdpportal.reg

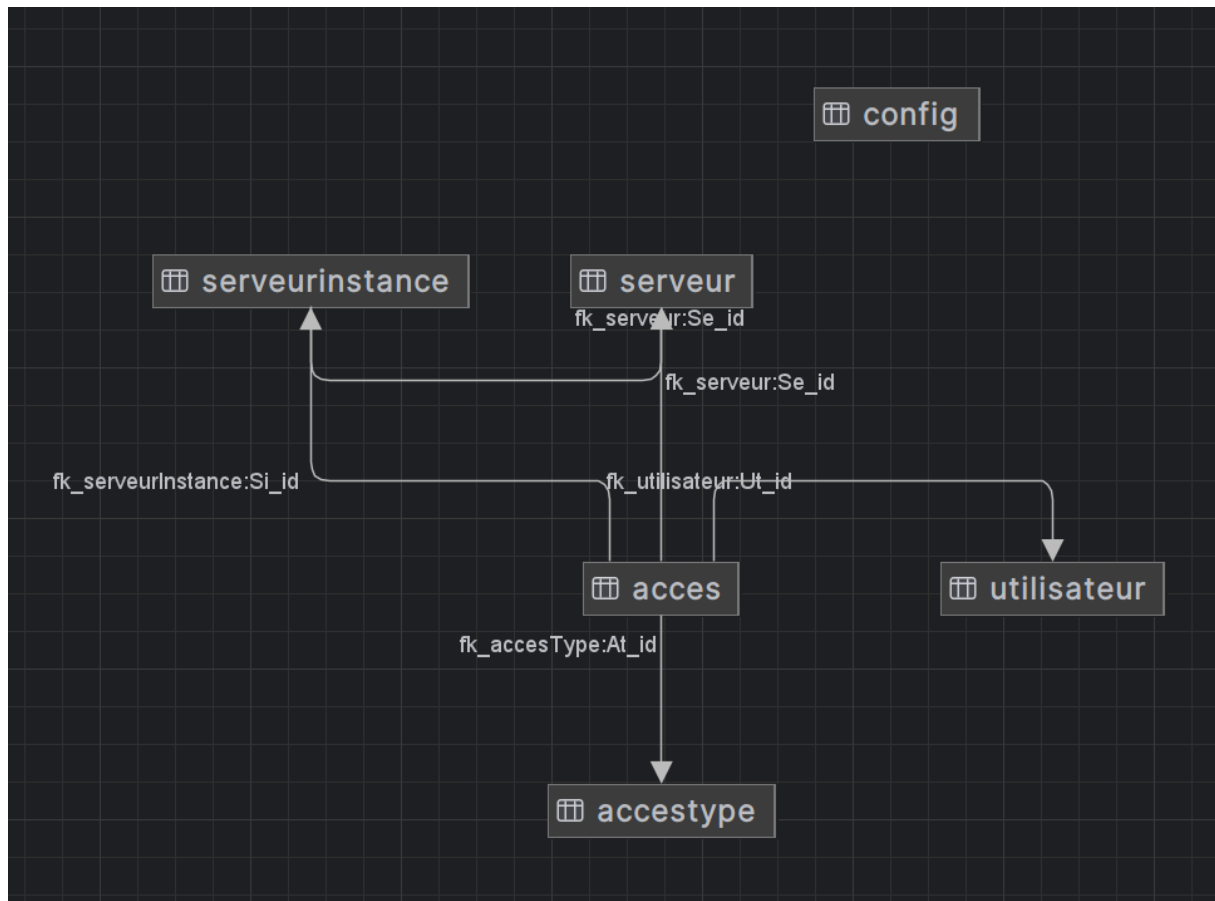
Problématique : PHP → exécutable bloqué par UAC/Permissions

Solution technique : Modification du registre Windows pour autoriser l'exécution d'.exe depuis contexte web

1.3 Installation

Voir document **PortailServeur-PremiereUtilisation.pdf**

2. Base de données



3. Code PHP

Le code PHP est séparé en 4 parties :

- **Index.php** : Point d'entrée unique + **routeur**
Analyse l'URL → redirige vers bon Controller
- **Models/** : dossier contenant 1 fichier par table dans la BDD, chacun de ces fichiers servira à interroger la BDD
- **Controllers/** : dossier contenant 1 fichier par page, gère la logique métier
- **Views/** : contient 1 dossier par page qui contient chacun 1 fichier php contenant la base HTML et 1 fichier JS qui contient tous les script nécessaire (Modals, AJAX, etc...)

Tout le CSS est quant à lui réunit dans un seul fichier **page.css**

3.1 Fonctionnement global

Prenons comme exemple la page **Tableau de Bord**, le fonctionnement de cette page est sensiblement le même sur toutes les autres.

Formulaires :

Submit HTML -> JS -> Controller PHP -> Model concerné -> MAJ BDD

```
document.querySelector(selectors: '#detailModal form').addEventListener(type: 'submit', listener: async e: Event => {
    e.preventDefault();

    const formData: FormData = new FormData(e.target);
    formData.append(name: 'action', DOM.get('modalAction').value);

    const submitBtn = e.target.querySelector(selectors: 'button[type=submit]');
    const errorDiv: HTMLElement = document.getElementById(elementId: 'formError');
    submitBtn.disabled = true;
    submitBtn.textContent = 'Traitement...';

    if (errorDiv) errorDiv.style.display = 'none';

    try {
        const res: Response = await fetch(input: '?url=dashboard', init: { method: 'POST', body: formData });
        const data = await res.json();

        if (data.success) {
            if (data.rdp_address) {
                window.location.href = 'rdpportal://${data.rdp_address}';
            }
            closeDetailModal();
            updateServers();
        } else {
            if (errorDiv) {
                errorDiv.textContent = data.error || 'Erreur inconnue';
                errorDiv.style.display = 'block';
            }
        }
    } catch (err) {
        if (errorDiv) {
            errorDiv.textContent = 'Erreur réseau';
            errorDiv.style.display = 'block';
        }
    } finally {
        submitBtn.disabled = false;
        submitBtn.textContent = 'Valider';
    }
}
```

Ici le JS, récupère les données du formulaire HTML et les envoie à cette méthode de **DashboardController.php**

```

#[NoReturn] private function createAccess(): void
{
    $serverId = (int)($_POST['server_id'] ?? 0);
    $comment = trim( string: $_POST['comment'] ?? '');
    $accessType = (int)($_POST['access_type'] ?? 0);
    $instanceIdRaw = $_POST['instance_id'] ?? '';
    $instanceId = $instanceIdRaw ? (int)$instanceIdRaw : null;

    // Vérification des champs obligatoires
    if ($serverId <= 0 || empty($comment) || $accessType <= 0) {
        echo json_encode([
            'success' => false,
            'error' => 'Tous les champs requis (raison, commentaire).'
        ]);
        exit;
    }

    $serverModel = new Server($this->pdo);
    $accessModel = new Access($this->pdo);
    $ok = $accessModel->createAccess(
        $comment,
        $serverId,
        $_SESSION['user_id'],
        $accessType,
        $instanceId
    );

    if (!$ok) {
        echo json_encode([
            'success' => false,
            'error' => 'Ce serveur est déjà verrouillé par un autre utilisateur.'
        ]);
        exit;
    }
}

```

La méthode récupère les données envoyées par le **JS**, et les transmet à la méthode **createAccess**. Que ça soit réussi ou non, le résultat est renvoyé sous forme d'un **JSON**.

Pourquoi du JSON et pas juste stocker en php ?

J'ai choisi de partir sur un renvoie en **JSON**, car je trouvais ça plus facile à gérer pour afficher les erreurs en direct sur le formulaire

Ce fonctionnement est la base de l'interface, pour toutes les actions il est répété juste les méthodes changent en fonction du contexte.


```
RewriteEngine On

RewriteCond %{HTTP_HOST} ^pictavinrt\.cluster023\.hosting\.ovh\.net$ [NC]
RewriteRule ^ServeursPictav/(.*)$ https://maintenance.pictav.info/$1 [R=301,L]

RewriteCond %{HTTP_HOST} ^pictavinrt\.cluster023\.hosting\.ovh\.net$ [NC]
RewriteRule ^(.*)$ https://maintenance.pictav.info/$1 [R=301,L]

RewriteCond %{HTTP_HOST} ^maintenance\.pictav\.info$ [NC]
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://maintenance.pictav.info/$1 [R=301,L]
```

Suite à la mise en place du SSO on a eu besoin de passer l'interface en **HTTPS**. Or on passe par une redirection et l'utilisateur connecté arrivait sur l'adresse **http non sécurisé**. Pour y remédier on utilise un fichier **.htaccess**, qui force la redirection vers l'adresse **HTTPS**.

4. Conclusion

Le fonctionnement général est expliqué dans ce document mais bien évidemment le code est commenté pour des explications plus précises sur le fonctionnement de tel ou tel fonction.